

EECS3311 Project – Stage 1 Design Report: StudyMuse

EECS3311 Software Design • Fall 2026 • Stage 1 Submission

1. Project Overview

StudyMuse is a Java desktop study assistant: a student uploads course materials, and an AI agent turns them into summaries, answers, flashcards, quizzes, and a personalized study plan. It is used through a JavaFX graphical interface (GUI) and a command-line interface (CLI) that share the same back end.

1.1 Problem and Motivation

University students receive large amounts of material per course: lecture slides, PDF readings, and personal notes. Turning that material into effective study activities is slow. Students must re-read everything to find answers, write their own flashcards and practice questions, and guess which topics they are weakest in.

General chatbots help only partly. They do not know the student's actual course content, they forget past sessions, and they cannot track quiz results over time. StudyMuse solves this by grounding every AI answer in the student's own uploaded materials and by remembering their performance.

1.2 Target Users

The primary users are university and college students preparing for tests and exams. Secondary users are self-learners studying from their own documents, such as online-course notes or textbooks.

1.3 What the Agent Does

The agent receives a study goal in natural language, such as "Quiz me on Chapter 3, then plan my week before the midterm." It then:

1. **Reasons and plans:** breaks the goal into ordered steps.
2. **Uses tools:** calls the system's own tools, such as searching the uploaded materials, generating a quiz, or reading quiz history.
3. **Retrieves:** finds the relevant passages of the student's materials before answering, so answers are based on course content rather than invented.
4. **Remembers:** keeps the conversation history and the student's quiz performance, and uses them in later requests.
5. **Executes multi-step tasks:** runs the steps in order, checks each result, and produces a final answer.

1.4 Why an AI Agent Is Appropriate

The core tasks require understanding natural language: summarizing a reading, writing good questions, grading a short written answer, and explaining a mistake. These cannot be done well by fixed rules, so an LLM is needed.

A single LLM call is not enough. Answers must be based on the student's documents (retrieval), study plans depend on past quiz scores (memory and tools), and requests like "quiz me, then plan my week" require several steps in the right order (planning). These are agent capabilities, combined with deterministic code for storage, scoring, and scheduling.

1.5 AI/LLM Models

Role	Model / Service	Use
Primary LLM	Anthropic Claude Sonnet, via the Anthropic Messages API	Planning, tool selection, generation, grading, explanations
Alternative LLM	OpenAI GPT model, via the OpenAI Chat Completions API	Interchangeable backup provider
Test double	MockLLMProvider (no network)	Returns fixed responses for automated testing

All three sit behind one `LLMProvider` interface, so the provider is chosen in a configuration file without changing other code.

1.6 How the AI Interacts with the Rest of the System

The LLM never touches files, the database, or the user interface directly. It only receives text prompts and returns text, which the system checks before acting on it. The system is organized in five layers:

1. **Presentation layer:** `MainWindow` (JavaFX GUI) and `StudyCLI` (command line). Both only collect input and display results.
2. **Controller layer:** `StudyAgentController` is the single entry point that both interfaces call.
3. **Agent layer:** `AgentPlanner` decides the steps; `ToolRegistry` holds the available `StudyTool` objects; `PromptBuilder` creates prompts; `LLMClient` sends them to the provider; `ResponseParser` validates the LLM's reply (for example, checks that a requested tool exists and its arguments are valid); `MemoryManager` stores conversation context.
4. **Service layer (deterministic):** services for materials, flashcards, quizzes, grading, analytics, and study plans. These contain the business logic and call the agent layer only when AI is needed.
5. **Persistence layer:** `CourseRepository` saves courses, materials, flashcards, quiz attempts, and plans as JSON files on the student's computer.

The agent works in a loop: the planner asks the LLM for the next action, the parser validates it, the tool registry runs the matching tool, and the tool result is added to the context. The loop ends when the LLM returns a final answer or a step limit (10 steps) is reached.

2. Feature Specifications

StudyMuse has 12 features: 3 deterministic, 4 AI-based, and 5 hybrid. Every feature is available in both the GUI and the CLI.

ID	Feature	Type
F01	Course and Material Upload	Deterministic
F02	Material Summarization	AI
F03	Question Answering over Materials	Hybrid
F04	Key Concept Extraction	AI
F05	Flashcard Generation	AI
F06	Flashcard Review with Spaced Repetition	Deterministic
F07	Quiz Generation	AI
F08	Quiz Grading and Feedback	Hybrid
F09	Weak-Topic Identification	Hybrid
F10	Personalized Study Plan Generation	Hybrid
F11	Progress Tracking Dashboard	Deterministic
F12	Agent Chat with Multi-Step Task Execution and Memory	Hybrid

F01 — Course and Material Upload

Field	Specification
Description	Creates a course and imports study materials (PDF, TXT, Markdown). Extracts the text, splits it into numbered chunks of about 500 words, and builds a keyword index used by all AI features for retrieval.
User Interaction	GUI: "New Course" button, then "Upload Material" with a file chooser. CLI: <code>course create "EECS3311"</code> , then <code>upload EECS3311 notes.pdf</code> .
Input	Course name; one or more files.
Output	Course with its list of materials, page count, and chunk count shown in the Course panel.
AI Involvement	Deterministic.

Field	Specification
Expected Workflow	1) User selects a file. 2) The system picks a reader for the file type. 3) Text is extracted. 4) Text is split into chunks and indexed. 5) Material is saved and listed.
Error/Alternative Cases	Unsupported file type: rejected with the list of supported types. Empty or scanned (image-only) PDF: warning that no text was found. Duplicate file name: user chooses to replace or keep both. File over 20 MB: rejected.

F02 — Material Summarization

Field	Specification
Description	Produces a structured summary of one material: an overview paragraph plus key points per section.
User Interaction	GUI: select a material, click "Summarize", choose length (short, medium, detailed). CLI: <code>summarize notes.pdf --length short</code> .
Input	Selected material; summary length.
Output	Summary displayed in the reading panel and saved with the material.
AI Involvement	AI.
Expected Workflow	1) System loads the material's chunks. 2) Long materials are summarized chunk by chunk, then combined. 3) Prompt is built with the length instruction. 4) LLM generates the summary. 5) Summary is validated, saved, and displayed.
Error/Alternative Cases	LLM unavailable or timeout: retry up to 2 times, then show an error and keep the material unchanged. Material has no text: summarization is disabled for it.

F03 — Question Answering over Materials

Field	Specification
Description	Answers a student's question using only the uploaded course materials and shows which material and chunk each part of the answer came from.
User Interaction	GUI: type a question in the "Ask" box of a course. CLI: <code>ask EECS3311 "What is the Observer pattern?"</code> .

Field	Specification
Input	Course; question text.
Output	Answer with source references (material name and chunk number).
AI Involvement	Hybrid: deterministic keyword search finds relevant chunks; the LLM writes the answer from them.
Expected Workflow	1) Agent receives the question. 2) Search tool retrieves the top 5 matching chunks. 3) Prompt is built with the question and chunks, instructing the LLM to answer only from them. 4) LLM answers. 5) Parser checks that cited sources exist. 6) Answer and sources are displayed.
Error/Alternative Cases	No relevant chunks found: agent says the materials do not cover the question instead of guessing. Answer cites a chunk that does not exist: the citation is removed and the answer is flagged as unverified.

F04 — Key Concept Extraction

Field	Specification
Description	Extracts the important concepts and terms from a material, each with a one-sentence definition. Concepts become the "topics" used by quizzes, weak-topic analysis, and study plans.
User Interaction	GUI: "Extract Concepts" button on a material; results shown as a list of concept cards. CLI: <code>concepts notes.pdf</code> .
Input	Selected material.
Output	List of concepts (name, definition, source chunk), saved with the course.
AI Involvement	AI.
Expected Workflow	1) Material chunks are loaded. 2) LLM is asked for concepts in a fixed JSON format. 3) Parser validates the JSON. 4) Duplicate concepts are merged. 5) Concepts are saved and displayed.
Error/Alternative Cases	Malformed JSON: the request is repeated once with a format reminder, then an error is shown. Zero concepts returned: user is told the material may be too short.

F05 — Flashcard Generation

Field	Specification
Description	Generates question-and-answer flashcards from a material or a chosen concept.
User Interaction	GUI: "Generate Flashcards", choose source and number of cards (5–50). Generated cards appear in an editable list for review before saving. CLI: <code>flashcards generate notes.pdf --count 20</code> .
Input	Material or concept; number of cards.
Output	A flashcard deck saved to the course.
AI Involvement	AI.
Expected Workflow	1) Relevant chunks are retrieved. 2) LLM generates cards in JSON format. 3) Parser validates each card (non-empty front and back). 4) Duplicate cards are removed. 5) User reviews, edits, or deletes cards. 6) Deck is saved.
Error/Alternative Cases	Count outside 5–50: rejected with a message. Fewer valid cards than requested: user is told how many were created. LLM failure: no partial deck is saved.

F06 — Flashcard Review with Spaced Repetition

Field	Specification
Description	Shows flashcards that are due today, and reschedules each card based on how well the student remembered it, so difficult cards return sooner.
User Interaction	GUI: "Review" mode shows one card, "Flip", then "Again / Hard / Good / Easy" buttons. CLI: <code>flashcards review EECS3311</code> , answering with 1–4.
Input	Deck; the student's rating for each card.
Output	Next review date for each card; session statistics (cards reviewed, percent correct).
AI Involvement	Deterministic (scheduling formula based on the SM-2 algorithm).
Expected Workflow	1) System loads cards due today. 2) Each card is shown and rated. 3) The scheduler calculates the next review interval. 4) The card is updated and saved. 5) Session statistics are recorded.

Field	Specification
Error/Alternative Cases	No cards due: show the date of the next due card. Session interrupted: ratings already given are kept.

F07 — Quiz Generation

Field	Specification
Description	Creates a practice quiz with multiple-choice and short-answer questions on chosen topics and difficulty.
User Interaction	GUI: "New Quiz" form with topics, number of questions (3–30), question types, and difficulty (easy, medium, hard). CLI: <code>quiz generate EECS3311 --topics "UML,Observer" --count 10 --difficulty medium</code> .
Input	Course; topics; question count; question types; difficulty.
Output	A saved quiz, ready to take.
AI Involvement	AI.
Expected Workflow	1) Chunks for the chosen topics are retrieved. 2) LLM generates questions in JSON with answers and topic labels. 3) Parser validates each question (for example, a multiple-choice question has exactly 4 options and one correct answer). 4) Invalid questions are regenerated once. 5) Quiz is saved and opened.
Error/Alternative Cases	Topic not found in materials: user is asked to choose another topic. Not enough valid questions after regeneration: quiz is created with fewer questions and the user is told.

F08 — Quiz Grading and Feedback

Field	Specification
Description	Grades a completed quiz and explains every wrong answer using the course materials.
User Interaction	GUI: take the quiz, click "Submit", see score and per-question feedback. CLI: <code>quiz take <quizId></code> , answering in the terminal.
Input	Quiz; the student's answers.

Field	Specification
Output	Total score, correct/incorrect status per question, and an explanation for each mistake. The attempt is saved.
AI Involvement	Hybrid: multiple-choice questions are graded deterministically; short answers are graded by the LLM against the model answer; explanations are AI-generated.
Expected Workflow	1) Multiple-choice answers are checked against the key. 2) Short answers are sent to the LLM with the model answer and a grading guide; it returns a score of 0, 0.5, or 1 with a reason. 3) Explanations are generated for wrong answers. 4) Score is calculated and the attempt is saved. 5) Progress statistics are updated.
Error/Alternative Cases	Blank answer: graded 0 without calling the LLM. LLM unavailable: multiple-choice is still graded; short answers are marked "pending" and can be graded later.

F09 — Weak-Topic Identification

Field	Specification
Description	Finds the topics where the student performs worst and explains why, based on quiz and flashcard history.
User Interaction	GUI: "Weak Topics" panel on the dashboard, with a ranked list and an explanation for each topic. CLI: <code>weak-topics EECS3311</code> .
Input	Course; stored quiz attempts and flashcard ratings.
Output	Ranked list of weak topics with accuracy percentage and a short diagnosis of the student's mistakes.
AI Involvement	Hybrid: accuracy per topic is calculated deterministically; the LLM analyzes the actual wrong answers to describe the pattern of mistakes.
Expected Workflow	1) Accuracy is calculated per topic. 2) Topics below 70% accuracy with at least 3 attempts are selected. 3) The LLM receives the wrong answers for those topics and describes the mistake pattern. 4) Results are displayed and saved for the study planner.
Error/Alternative Cases	Not enough data (fewer than 3 attempts on any topic): user is told to take more quizzes first. No topic below 70%: user is congratulated and shown their lowest topic.

F10 — Personalized Study Plan Generation

Field	Specification
Description	Creates a day-by-day study schedule up to an exam date, giving more time to weak topics.
User Interaction	GUI: "Study Plan" form with exam date, hours available per day, and topics to include; the plan is shown in a calendar view. CLI: <code>plan EECS3311 --exam 2026-10-20 --hours 2</code> .
Input	Course; exam date; daily hours; topics.
Output	A study plan of daily sessions (topic, activity such as review flashcards or take quiz, and duration).
AI Involvement	Hybrid: the LLM drafts the plan using weak-topic results; deterministic code validates it.
Expected Workflow	1) Agent gets the weak topics (F09) and the list of concepts. 2) LLM drafts the plan. 3) A validator checks that no day exceeds the available hours, every chosen topic appears, and all dates are before the exam. 4) If the check fails, the LLM is asked to fix the listed problems (maximum 2 attempts). 5) Plan is saved and displayed.
Error/Alternative Cases	Exam date in the past: rejected. Too little time for all topics: plan prioritizes weak topics and lists those left out. Validation still fails after 2 attempts: a simple deterministic plan (equal time per topic) is created instead.

F11 — Progress Tracking Dashboard

Field	Specification
Description	Shows the student's progress per course: quiz scores over time, flashcards mastered, study time, and study-plan completion.
User Interaction	GUI: "Dashboard" tab with charts that refresh automatically after each quiz or review session. CLI: <code>progress EECS3311</code> prints a text summary.
Input	Course; stored session and attempt records.
Output	Charts and statistics: average score, score trend, cards mastered, minutes studied, plan tasks completed.

Field	Specification
AI Involvement	Deterministic.
Expected Workflow	1) Records are loaded. 2) Statistics are calculated. 3) Dashboard is displayed. 4) When a quiz or review session ends, the dashboard receives an update notification and refreshes.
Error/Alternative Cases	No activity yet: an empty-state message suggests a first step. Corrupted record file: the bad record is skipped and the user is warned.

F12 — Agent Chat with Multi-Step Task Execution and Memory

Field	Specification
Description	A chat where the student gives requests in plain language. The agent plans the steps, calls the right tools (search, quiz, flashcards, weak topics, study plan), and remembers earlier messages in the session and key facts across sessions (such as the exam date).
User Interaction	GUI: "Agent Chat" panel showing the conversation and each step the agent takes. CLI: chat, which starts an interactive session.
Input	A natural-language request, for example "Quiz me on my weakest topic and add the mistakes to my flashcards."
Output	The final answer plus a visible list of steps executed (tool name and short result).
AI Involvement	Hybrid: the LLM plans and selects tools; deterministic code validates and runs the tools.
Expected Workflow	1) The request and relevant memory are added to the prompt along with the list of available tools. 2) LLM returns either a tool call or a final answer. 3) Parser validates the tool call (tool exists, arguments valid). 4) Tool is executed and its result added to the context. 5) Steps 2–4 repeat until a final answer is produced, with a maximum of 10 steps. 6) Answer and steps are displayed; memory is updated.
Error/Alternative Cases	Unknown tool or invalid arguments: the error is returned to the LLM so it can correct itself. A tool fails: the agent reports the failure and offers an alternative instead of inventing a result. Request is ambiguous (for example, no course given): the agent asks a clarifying question. Step limit reached: the agent stops and summarizes what was completed.

3. UML Class Diagram

The StudyMuse class model has about 60 classes in 9 packages. For readability it is shown as four views of the same model: CD-1 gives the overview, and CD-2 to CD-4 show each layer with full attributes, methods, and multiplicities. A class that appears in more than one view is the same class.

Notation used (following the EECS3311 UML lecture): + public, - private, # protected; underlined members are static; bold names are concrete classes; «interface» and «abstract» mark interfaces and abstract classes, whose names are in italics; stereotypes such as «Facade» mark the class's role in a design pattern. All diagrams in this report were drawn in UMLet; the .uxf source files are in the repository's diagrams folder.

Line	Meaning
Solid line, open arrow	Association (the class holds a reference)
Filled diamond	Composition (the part cannot exist without the whole)
Hollow diamond	Aggregation (the whole groups parts that can exist alone)
Dashed line, open arrow	Dependency (uses, creates, or returns)
Dashed line, hollow triangle	Realization (implements an interface)
Solid line, hollow triangle	Inheritance (extends a class)

CD-1 — Architecture Overview

The system has six layers. The GUI and CLI talk only to the controller; the controller delegates to deterministic services or to the agent; only the LLM package talks to an external AI service.

CD-1 StudyMuse — Architecture Overview (key classes per layer)

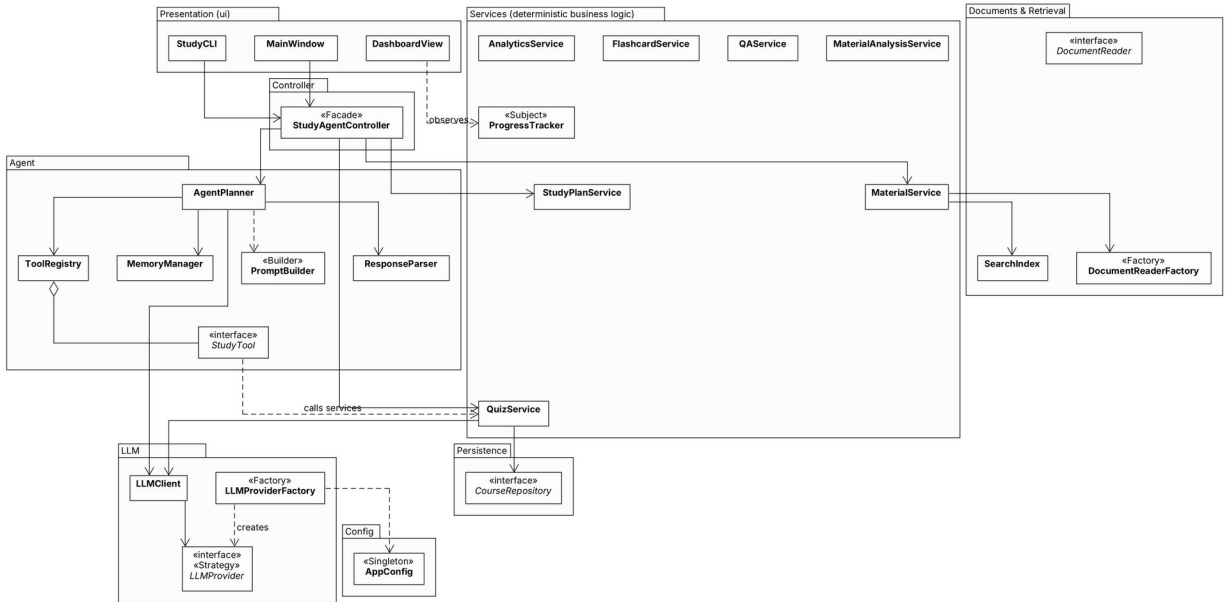


Figure: CD-1 Architecture overview

CD-2 — Presentation, Controller and Services

StudyAgentController is the single entry point for both interfaces. Each service owns one area of business logic. QuizService delegates grading to interchangeable GradingStrategy objects, and ProgressTracker notifies the dashboard when progress changes.

CD-2 Presentation, Controller and Service Layer

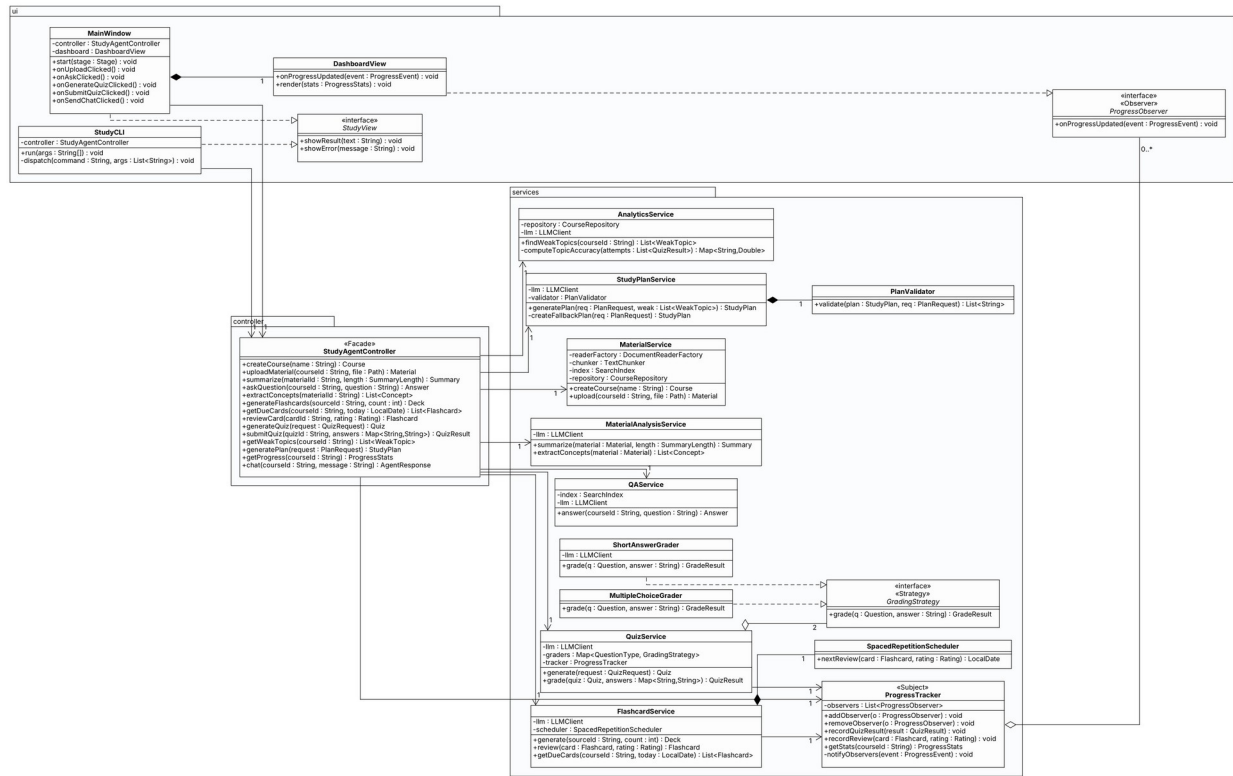


Figure: CD-2 Presentation, controller and services

CD-3 — Agent Core, Tools and LLM

AgentPlanner runs the agent loop for up to 10 steps. It builds prompts with PromptBuilder, sends them through LLMClient, validates replies with ResponseParser, and runs tools through ToolRegistry. The LLM provider is chosen by LLMProviderFactory from the AppConfig settings and can be wrapped with retry and logging decorators.

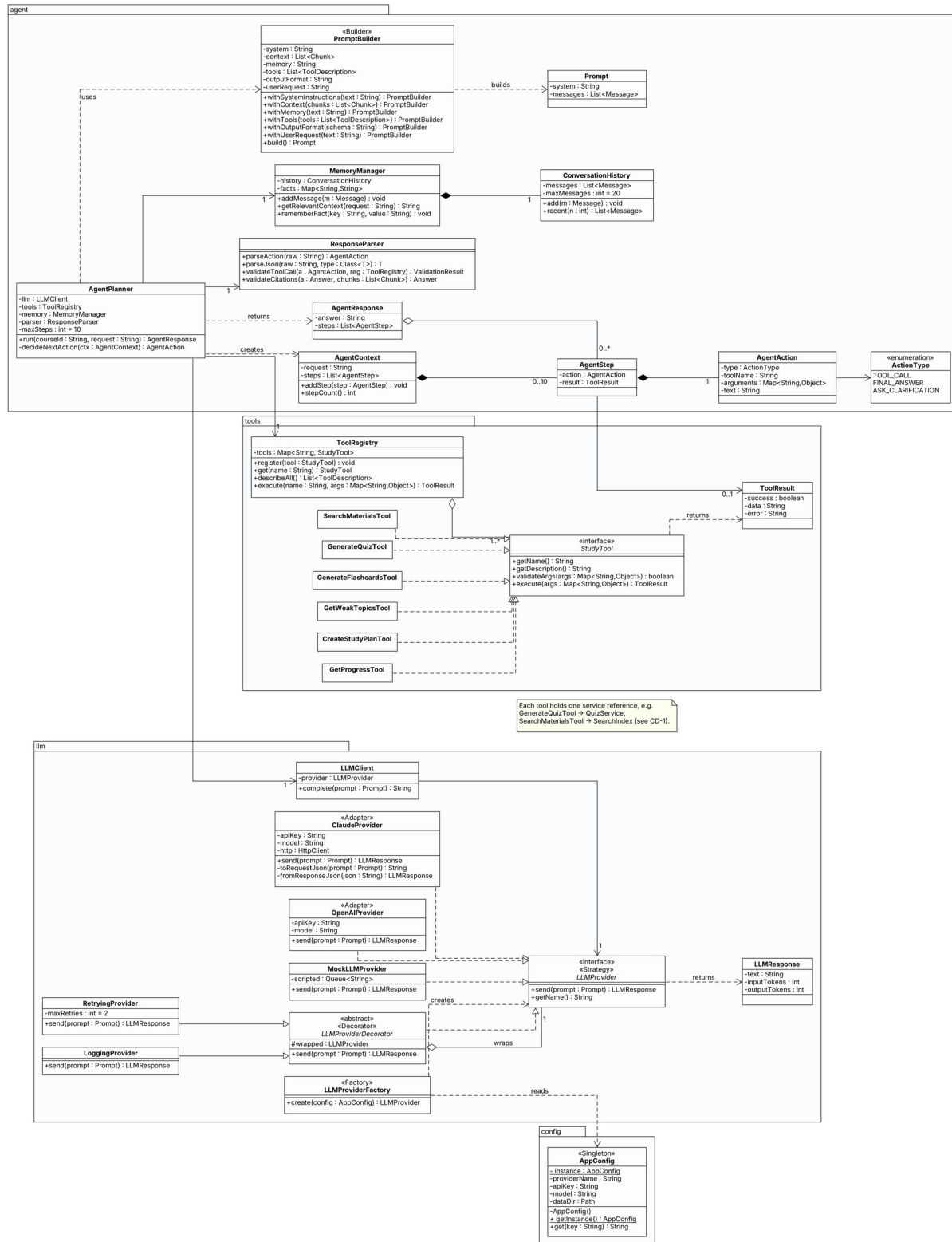


Figure: CD-3 Agent core, tools and LLM

CD-4 — Documents, Domain Model and Persistence

Uploaded files are read by a DocumentReader chosen by a factory, split into Chunks, and indexed for search. The domain model is centred on Course, which owns materials, concepts, decks, quizzes, and a study plan. CourseRepository is an interface, so storage can change without affecting services.

CD-4 Documents & Retrieval, Domain Model and Persistence

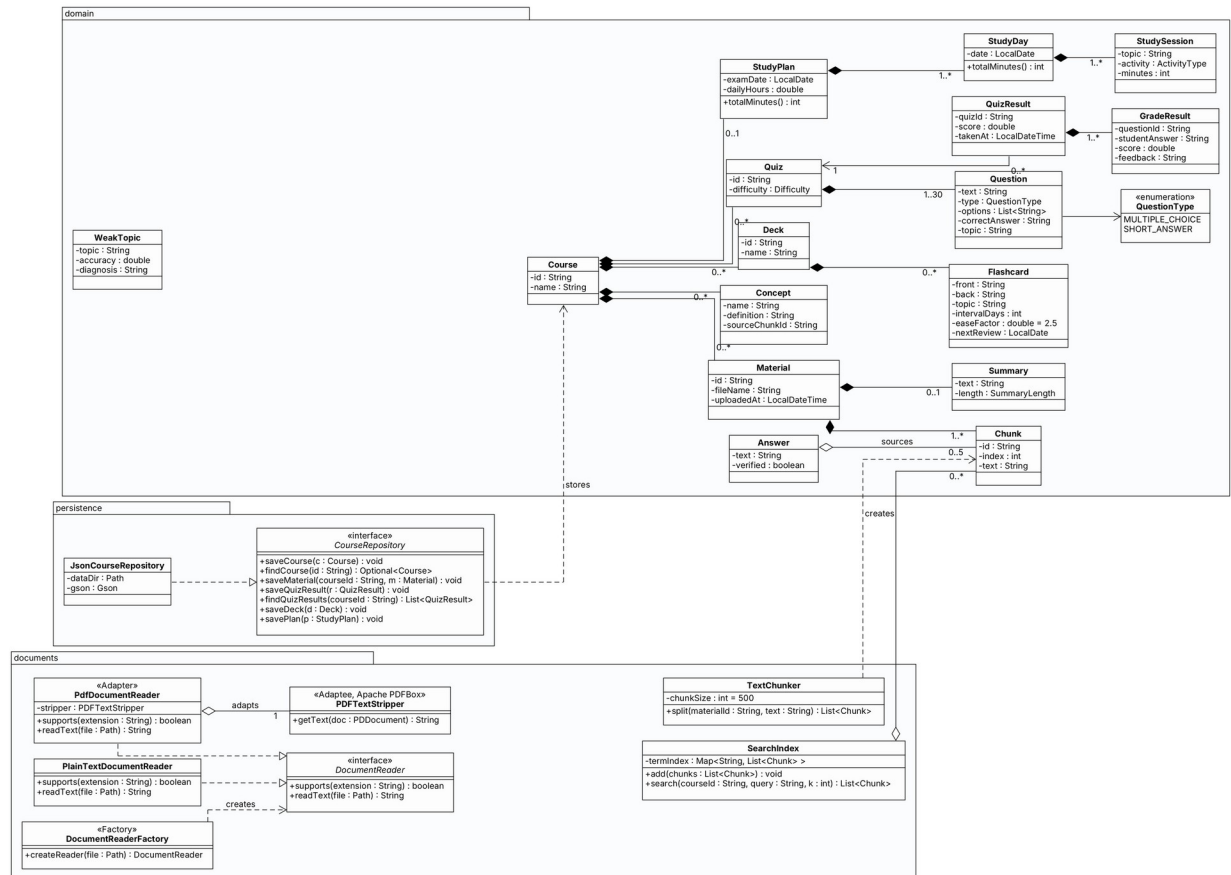


Figure: CD-4 Documents, domain model and persistence

3.1 Design Principles Applied

The design applies a small set of principles consistently so that the system stays implementable in Stage 2 and testable in Stage 3.

Separation of concerns / layered architecture. The system is split into presentation (ui), controller, agent, deterministic services, LLM access, documents & retrieval, domain, and persistence. Each layer has one job: the ui renders, the controller orchestrates, services implement feature logic, and the agent layer plans. A class never reaches across more than one layer boundary.

Single Responsibility Principle. Each service owns exactly one feature area: MaterialService owns ingestion, QuizService owns quiz generation and grading, FlashcardService owns decks and reviews, AnalyticsService owns statistics, StudyPlanService owns planning, and QAService owns grounded question answering. If a feature's rules change, only its service changes.

Open/Closed Principle. Variation points are captured as interfaces so new variants can be added without modifying existing code: new question types plug in as GradingStrategy implementations, new LLM vendors as LLMProvider implementations, and new file types as DocumentReader implementations.

Dependency inversion. High-level classes depend on abstractions, not concretions: services depend on the LLMProvider and DocumentReader interfaces, the UI depends on the StudyAgentController facade, and tools depend on service interfaces. Concrete classes are created only inside factories.

Deterministic core, AI at the edges. Everything that must be exact (grading multiple-choice answers, scheduling reviews, validating plans, parsing tool calls) is deterministic code. The LLM is used only where judgment over text is required (summaries, answers, feedback, diagnoses, plans). Every AI output passes through a validator or parser before it affects persisted state.

Fail-safe defaults. The design assumes the LLM can fail: RetryingProvider retries transient errors, ShortAnswerGrader degrades to 'pending' when the LLM is down, StudyPlanService falls back to a deterministic plan after two failed validation rounds, and the agent loop is capped at 10 steps so it always terminates.

Principle	Where it appears
Separation of concerns	UI only displays; services hold business rules; the agent package holds AI reasoning; only LLM calls external APIs.
Abstraction and interfaces	LLMProvider, StudyTool, DocumentReader, GradingStrategy, CourseRepository, ProgressObserver.
Dependency inversion	Services depend on LLMProvider and CourseRepository interfaces, not on Claude or JSON-file classes.
Polymorphism	ToolRegistry runs any StudyTool through the same execute() call; QuizService grades any question through GradingStrategy.
Encapsulation	All fields are private; the LLM's raw text is only exposed after ResponseParser validates it.
Low coupling, high cohesion	The two interfaces share one facade; each service has one responsibility; the LLM never accesses files or the UI.

4. Design Patterns

StudyMuse uses 8 design patterns, each solving a specific problem in the design. Six come from the course's creational and structural pattern lectures (Facade, Adapter, Decorator, Factory Method, Singleton, Builder); Strategy and Observer are behavioural patterns from the project's list of allowed patterns.

#	Pattern	Category	Main classes	Problem solved
P1	Facade	Structural	StudyAgentController	One entry point for both GUI and CLI
P2	Strategy	Behavioural	LLMProvider, GradingStrategy	Swap AI providers and grading methods
P3	Adapter	Structural	PdfDocumentReader, ClaudeProvider	Fit external libraries and APIs to our interfaces
P4	Decorator	Structural	RetryingProvider, LoggingProvider	Add retry and logging to any provider
P5	Factory Method	Creational	DocumentReaderFactory, LLMProviderFactory	Create the right object without if/else in services
P6	Singleton	Creational	AppConfig	One shared configuration
P7	Builder	Creational	PromptBuilder	Assemble prompts with many optional parts
P8	Observer	Behavioural	ProgressTracker, DashboardView	Refresh the dashboard without services knowing the UI

P1 — Facade

Problem addressed: StudyMuse has about ten services plus an agent layer and an LLM layer. If the GUI (MainWindow, DashboardView) and the CLI (StudyCLI) each talked to all of these directly, both front-ends would need to know the full subsystem: which service to call, in what order, and how to combine results. Feature logic would be duplicated in two places and any change would require edits in both.

Classes involved and roles played: StudyAgentController plays the Facade: it exposes one simple method per user-visible feature (createCourse, uploadMaterial, summarize, askQuestion, extractConcepts, generateFlashcards, reviewCard, generateQuiz, submitQuiz, getWeakTopics, generatePlan, getProgress, chat). MainWindow, DashboardView and StudyCLI are the clients and only ever call the facade. MaterialService, MaterialAnalysisService, QAService, QuizService, FlashcardService, AnalyticsService, StudyPlanService, ProgressTracker and AgentPlanner are the subsystem classes that do the real work behind the facade's methods.

Why this pattern is appropriate: The two front-ends must offer identical behaviour: a quiz generated from the CLI must be gradable from the GUI. A single facade guarantees one orchestration per feature, implemented once and shared. It also hides the layering (agent vs. deterministic services vs. LLM) from the UI, which only needs feature-level operations.

What would be harder without it: MainWindow would hold references to ten or more services and duplicate their call sequences in StudyCLI. Adding a feature would mean writing the orchestration twice and keeping the two copies consistent. Unit testing a feature flow would require wiring the whole subsystem instead of calling one facade method.

Aspect	Explanation
Problem	Both the GUI and the CLI must reach all 12 features, which involve 8 services and the agent. Without one entry point, each interface would depend on every service and repeat the same coordination code.
Participants and roles	StudyAgentController = Facade. MaterialService, QAService, QuizService, FlashcardService, AnalyticsService, StudyPlanService, ProgressTracker, AgentPlanner = subsystem classes. MainWindow, StudyCLI = clients.
Why appropriate	The facade offers one simple method per feature (for example <code>generateQuiz()</code>), so the GUI and CLI behave identically and stay thin.
Without it	Both interfaces would be coupled to every service; any service change would break two places, and GUI and CLI behaviour could drift apart.

P2 — Strategy

Problem addressed: Two independent dimensions of the system vary by algorithm. Quiz grading depends on question type: multiple-choice answers are checked deterministically against the key, while short answers need LLM judgment against a model answer. LLM access depends on vendor: Claude, OpenAI and a mock provider all send prompts differently. Hard-coding these choices leads to branching logic that grows with every new type or vendor.

Classes involved and roles played: GradingStrategy is the strategy interface (`grade(question, answer)`); MultipleChoiceGrader and ShortAnswerGrader are concrete strategies; QuizService is the context and holds a Map from QuestionType to GradingStrategy. LLMProvider is a second strategy interface (`send(prompt)`); ClaudeProvider, OpenAIProvider and MockLLMProvider are concrete strategies; LLMClient is the context holding the configured provider.

Why this pattern is appropriate: Both variation points are genuine families of interchangeable algorithms selected at run time (question type per question; vendor per configuration). New variants — e.g. a TrueFalseGrader or a new vendor — are added as one new class with zero changes to QuizService

or LLMClient (Open/Closed Principle). MockLLMProvider also makes offline and deterministic testing possible, which Stage 3 depends on.

What would be harder without it: QuizService.grade would contain a switch on question type and LLMClient a switch on vendor name, edited every time a type or vendor is added. Testing grading logic would require a live LLM connection, and swapping vendors would mean touching every call site instead of one configuration value.

Aspect	Explanation
Problem	(a) The AI provider must be interchangeable: Claude in normal use, OpenAI as a backup, and a mock with no network for automated tests. (b) Grading works differently for multiple-choice and short-answer questions.
Participants and roles	(a) LLMProvider = Strategy interface; ClaudeProvider, OpenAIProvider, MockLLMProvider = concrete strategies; LLMClient = context. (b) GradingStrategy = Strategy interface; MultipleChoiceGrader, ShortAnswerGrader = concrete strategies; QuizService = context, choosing a grader by QuestionType.
Why appropriate	Each algorithm sits in its own class behind one interface and is selected at runtime, which supports Stage 3 testing with the mock provider.
Without it	if (provider.equals("claude")) checks would spread across the code; tests would need real, slow, paid API calls; adding a new question type would mean editing QuizService.

P3 — Adapter

Problem addressed: Third-party APIs do not match the interfaces our design wants. The Anthropic and OpenAI HTTP APIs speak in vendor-specific request/response JSON, not in our Prompt/LLMResponse types. Apache PDFBox exposes PDFTextStripper.getText(PDDocument), not our DocumentReader.readText(Path). We cannot change those third-party classes, but we do not want their shapes leaking into our services.

Classes involved and roles played: ClaudeProvider and OpenAIProvider are adapters from the vendor APIs (the adaptees) to our LLMProvider target interface: toRequestJson translates our Prompt into the vendor's format and fromResponseJson translates the reply back into LLMResponse.

PdfDocumentReader is an adapter from PDFTextStripper (the adaptee, labelled «Adaptee, Apache PDFBox») to our DocumentReader target interface (supports/readText). PlainTextDocumentReader implements DocumentReader directly since plain text needs no adaptation.

Why this pattern is appropriate: Our services (QAService, QuizService, MaterialService, ...) depend only on our own interfaces, so a vendor API change — or a decision to replace PDFBox with another PDF library — is contained inside one adapter class. The rest of the system never sees vendor types.

What would be harder without it: Vendor-specific JSON handling would be scattered across every service that calls the LLM, and PDFBox types would appear in MaterialService. Upgrading or replacing either library would require hunting down every usage instead of rewriting a single adapter.

Aspect	Explanation
Problem	External code has interfaces we do not control: Apache PDFBox's PDFTextStripper and the Anthropic and OpenAI HTTP/JSON APIs. The rest of the system expects DocumentReader and LLMProvider.
Participants and roles	DocumentReader = Target; PdfDocumentReader = Adapter; PDFTextStripper = Adaptee. LLMProvider = Target; ClaudeProvider, OpenAIProvider = Adapters; the vendor web APIs = Adaptees. These are object adapters (composition), as in the lecture.
Why appropriate	Each adapter translates one external interface into ours, so library and vendor details stay inside one class.
Without it	PDFBox objects and vendor JSON formats would leak into services; changing the PDF library or AI vendor would require changes throughout the code.

P4 — Decorator

Problem addressed: Two behaviours must be addable to any LLM provider, in any combination: retrying transient failures (up to 2 retries) and logging token usage. Subclassing each provider for each combination would explode the class count (RetryingClaudeProvider, LoggingOpenAIProvider, ...), and baking retry/logging into the providers themselves would duplicate the logic and prevent turning it off.

Classes involved and roles played: LLMProvider is the component interface. LLMProviderDecorator is the abstract decorator: it implements LLMProvider, holds a wrapped LLMProvider, and its send delegates to the wrapped instance. RetryingProvider and LoggingProvider are concrete decorators that add retry logic and token logging around the delegated call. LLMProviderFactory stacks them, e.g. LoggingProvider(RetryingProvider(ClaudeProvider)).

Why this pattern is appropriate: Retry and logging are orthogonal to which vendor is used: they are cross-cutting concerns, not vendor traits. Decoration composes them at run time from the configuration, giving all combinations from three small classes, and a future concern (e.g. response caching) becomes one more decorator instead of N new subclasses.

What would be harder without it: Every provider would reimplement its own retry loop and logging, or the design would need a subclass per combination. Changing the retry policy would mean editing every provider, and tests could not easily run a provider with and without retry behaviour.

Aspect	Explanation
Problem	Every provider needs extra behaviour: retry on timeouts (features F02 and F08 require up to 2 retries) and logging of prompts and responses (needed for debugging and Stage 3 agent testing). This should work for any provider without changing it.
Participants and roles	LLMProvider = Component; ClaudeProvider etc. = concrete components; LLMProviderDecorator = abstract Decorator holding a wrapped provider; RetryingProvider, LoggingProvider = concrete decorators.
Why appropriate	Decorators stack at runtime, for example <code>new LoggingProvider(new RetryingProvider(new ClaudeProvider()))</code> , and each adds one responsibility.
Without it	Retry and logging code would be copied into each provider, or subclasses would multiply (<code>RetryingClaudeProvider</code> , <code>LoggingRetryingOpenAIPProvider</code> , and so on).

P5 — Factory Method

Problem addressed: Choosing which concrete class to instantiate requires conditional logic that should not leak into business code: the LLM provider depends on a configuration value (Claude vs OpenAI vs mock, plus which decorators to stack); the document reader depends on the file extension (PDF vs plain text vs Markdown). If services contain these if/else chains, every new option forces edits in every service that creates these objects.

Classes involved and roles played: LLMProviderFactory is the factory for the LLM side: `create(config)` reads `AppConfig` and returns a fully assembled LLMProvider (concrete provider wrapped in the configured decorators). DocumentReaderFactory is the factory for ingestion: `createReader(file)` inspects the extension and returns the matching DocumentReader. Clients (LLMClient, MaterialService) receive only the interface types.

Why this pattern is appropriate: All creation knowledge — which extensions map to which reader, how the provider/decorator stack is assembled — lives in exactly one place per family. Services stay decoupled from concrete classes, which also keeps the Adapter and Decorator patterns effective: clients never name the classes those patterns introduce.

What would be harder without it: MaterialService and LLMClient would each embed the selection logic. Adding Markdown support or a new vendor would require finding and editing every creation site, with a real risk of missing one and getting inconsistent behaviour between features.

Aspect	Explanation
Problem	The class to create depends on runtime information: the file extension decides the document reader, and the configuration decides the AI provider and its decorators.
Participants and roles	<code>DocumentReaderFactory.createReader(file) = factory;</code> <code>DocumentReader = product interface; PdfDocumentReader,</code> <code>PlainTextDocumentReader = concrete products.</code> <code>LLMProviderFactory.create(config) = factory; LLMProvider =</code> <code>product; providers and decorators = concrete products.</code>
Why appropriate	Creation logic is in one place; services ask for "a reader" or "a provider" and never use <code>new</code> on concrete classes.
Without it	<code>MaterialService</code> would contain an <code>if/else</code> on file extensions; supporting a new format such as <code>.docx</code> would mean editing service code instead of adding one class.

P6 — Singleton

Problem addressed: Configuration (provider name, API key, model, data directory) is needed in many places — factories, services, the agent — but it must be loaded exactly once and stay consistent everywhere. Passing a config object through the constructors of ~60 classes would add noise to every signature, and letting each class load its own copy risks disagreement (e.g. two different data directories).

Classes involved and roles played: `AppConfig` is the singleton: a private constructor, a static instance field, and a static `getInstance()` accessor plus `get(key)`. Every class that needs configuration calls `AppConfig.getInstance()`; no other instance can exist.

Why this pattern is appropriate: Application configuration is genuinely global, read-only after startup, and needed pervasively — the textbook case for a singleton. One load point also makes startup validation (missing API key, bad data directory) happen once, before any feature runs.

What would be harder without it: Constructors across the codebase would each take a config parameter, or each class would re-read config files itself. Tests would struggle to substitute configuration, and a partially-initialised config in one corner of the app could cause subtle, hard-to-reproduce bugs.

Aspect	Explanation
Problem	Settings (provider name, model, API key, data folder) are read from one file and needed by several classes. They must be loaded once and be identical everywhere.

Aspect	Explanation
Participants and roles	AppConfig = Singleton, with a private constructor, a static instance field, and a static getInstance() method.
Why appropriate	There is exactly one configuration per running application. Singleton is used only for configuration, not business objects, so services stay easy to test.
Without it	The file would be re-read in many places, settings could become inconsistent, and the API key would be passed through many constructors.

P7 — Builder

Problem addressed: An LLM prompt is assembled from up to seven optional parts: system instructions, retrieved chunks, conversation memory, tool descriptions, output-format schema, and the user request. A constructor or method taking all of these would be unreadable (which argument was null again?), and different features need different combinations — Q&A needs chunks plus citation rules, agent chat needs memory plus tools, plan generation needs a JSON schema.

Classes involved and roles played: PromptBuilder is the builder: withSystemInstructions, withContext, withMemory, withTools, withOutputFormat and withUserRequest each add one part and return the builder for chaining; build() produces the immutable Prompt (the product). The calling service (QAService, AgentPlanner, StudyPlanService, ...) plays the director role, deciding which parts a particular prompt needs.

Why this pattern is appropriate: The fluent calls make every prompt-construction site self-documenting: a reader can see exactly which parts the Q&A prompt includes versus the agent prompt. Optional parts are simply omitted rather than passed as null, and Prompt itself stays immutable once built, so it can be safely logged and passed to the LLM client.

What would be harder without it: Prompt would need a many-argument constructor or a bag of setters with no clear contract about which combinations are valid. Prompt-building logic would be duplicated at each of the half-dozen call sites, and changing the prompt structure (e.g. adding tool descriptions) would require editing every site.

Aspect	Explanation
Problem	A prompt has many optional parts: system instructions, retrieved chunks, memory, tool descriptions, output format, and the user request. Each feature needs a different combination.
Participants and roles	PromptBuilder = Builder with step methods (withContext(), withTools(), ...) and build(); Prompt = Product; the services and AgentPlanner = Directors that decide which steps to call.

Aspect	Explanation
Why appropriate	Prompts are built step by step in a readable chain, and <code>build()</code> can check that required parts (such as the user request) are present.
Without it	A Prompt constructor with six parameters, many of them <code>null</code> , or string concatenation duplicated in every service, producing inconsistent prompts.

P8 — Observer

Problem addressed: When a quiz is graded or a review session ends, the progress dashboard must refresh — but QuizService and FlashcardService should not know that a dashboard exists. Hard-coding a call to DashboardView inside the services would couple business logic to the GUI, make the services untestable without a UI, and force edits to every service each time a new view (e.g. a CLI progress summary) is added.

Classes involved and roles played: ProgressTracker is the subject: it keeps a list of ProgressObserver instances and offers addObserver, removeObserver and notifyObservers. ProgressObserver is the observer interface (onProgressUpdated). DashboardView implements it and re-renders on notification. QuizService and FlashcardService only call recordQuizResult/recordReview on the tracker; the tracker notifies whoever is registered.

Why this pattern is appropriate: Progress display is a classic one-to-many dependency: one stream of events (quiz results, reviews), many potential displays (GUI dashboard now, CLI summary later). The services stay UI-agnostic and unit-testable, and new views subscribe without any change to existing code.

What would be harder without it: Services would import UI classes and call them directly, so running the quiz logic headless (tests, CLI) would drag in JavaFX. Each new display would require editing QuizService, FlashcardService and any future producer of progress events.

Aspect	Explanation
Problem	The dashboard (F11) must refresh when a quiz is graded or a flashcard session ends, but services must not depend on the GUI, because the CLI has no dashboard.
Participants and roles	ProgressTracker = Subject (addObserver(), notifyObservers()); ProgressObserver = Observer interface; DashboardView = concrete observer; QuizService, FlashcardService = classes that report events to the subject.
Why appropriate	Services report progress to the tracker, and any number of views can subscribe without the services knowing who is listening.

Aspect	Explanation
Without it	QuizService would call DashboardView directly, making the service layer depend on the GUI and breaking the CLI.

5. Use-Case Diagram and Descriptions

The use-case diagram below shows the twelve use cases of StudyMuse, one per feature (UC01–UC12), the primary actor (Student), and the external LLM Service actor used by every AI or hybrid feature. Each use case is described in full after the diagram.

UC-1 StudyMuse — Use-Case Diagram

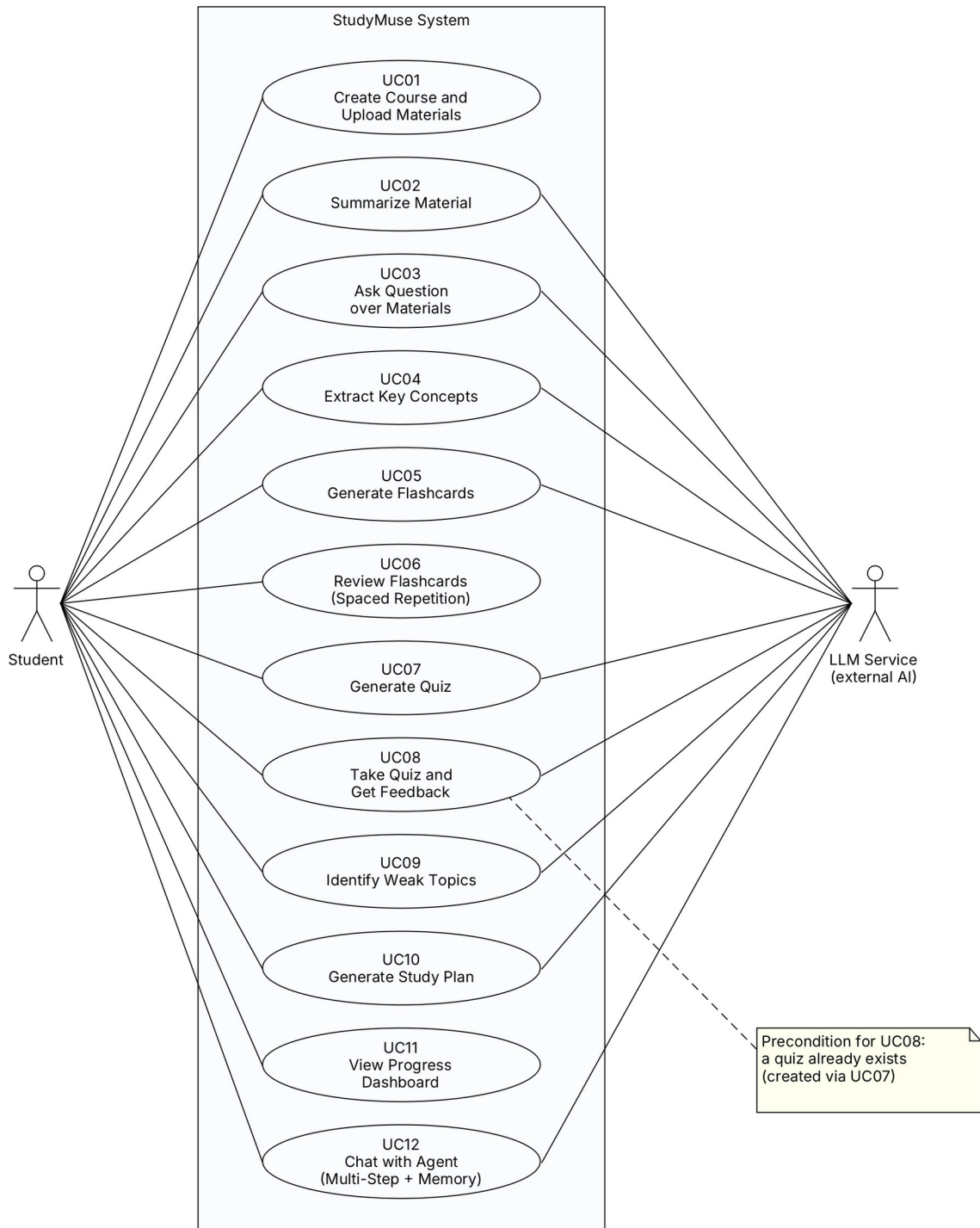


Figure UC-1 — StudyMuse use-case diagram (UC01-UC12, Student and LLM Service actors).

UC01 — Create Course and Upload Materials

Field	Description
Use-Case ID	UC01
Name	Create Course and Upload Materials
Actor(s)	Student
Goal	Get course materials into the system so that every AI feature can use them.
Preconditions	The application is running. A course may or may not exist yet.
Trigger	Student clicks "New Course" / "Upload Material" (GUI) or runs course create / upload (CLI).
Main Success Scenario	1. Student enters a course name; the system creates the course. 2. Student selects one or more files to upload. 3. The system picks a document reader based on the file extension and extracts the text. 4. The text is split into numbered chunks of about 500 words and added to the keyword index. 5. The material is saved; the Course panel shows the material with page and chunk counts.
Alternative / Exception Flows	• Unsupported file type: upload rejected, supported types listed; nothing is saved. • Empty or image-only (scanned) PDF: warning that no text was found; material saved without chunks. • Duplicate file name: student chooses to replace the existing file or keep both. • File over 20 MB: rejected with an explanatory message.
Postconditions	Course and materials are persisted as JSON; chunks are indexed and searchable.
Related Feature(s)	F01

UC02 — Summarize Material

Field	Description
Use-Case ID	UC02
Name	Summarize Material
Actor(s)	Student, LLM Service
Goal	Produce a structured summary of one material: an

	overview paragraph plus key points per section.
Preconditions	A material with extractable text exists in the course.
Trigger	Student selects a material, clicks "Summarize" and chooses a length (GUI), or runs summarize (CLI).
Main Success Scenario	1. The system loads the material's chunks. 2. Long materials are summarized chunk by chunk; the partial summaries are then combined. 3. A prompt is built with the length instruction (short / medium / detailed). 4. The LLM generates the summary. 5. The summary is validated, saved with the material, and displayed.
Alternative / Exception Flows	• LLM timeout or failure: retried up to 2 times, then an error is shown and the material is left unchanged. • Material has no text: summarization is disabled for that material.
Postconditions	The summary is saved with the material and shown in the reading panel.
Related Feature(s)	F02

UC03 — Ask Question over Materials

Field	Description
Use-Case ID	UC03
Name	Ask Question over Materials
Actor(s)	Student, LLM Service
Goal	Answer the student's question using only the uploaded materials, showing the source of each part.
Preconditions	The course contains indexed materials.
Trigger	Student types a question in the "Ask" box (GUI) or runs ask (CLI).
Main Success Scenario	1. The agent receives the question. 2. The search tool retrieves the top 5 matching chunks from the index. 3. A prompt is built from the question and the chunks, instructing the LLM to answer only from them. 4. The LLM produces the answer. 5. The parser checks that every cited chunk actually exists. 6. The answer is displayed with source references

	(material name and chunk number).
Alternative / Exception Flows	• No relevant chunks found: the agent states the materials do not cover the question instead of guessing. • A citation refers to a chunk that does not exist: the citation is removed and the answer is flagged unverified.
Postconditions	None persisted; the answer is transient and shown with its sources.
Related Feature(s)	F03

UC04 — Extract Key Concepts

Field	Description
Use-Case ID	UC04
Name	Extract Key Concepts
Actor(s)	Student, LLM Service
Goal	Extract the important concepts and terms from a material, each with a one-sentence definition.
Preconditions	A material with extractable text exists.
Trigger	Student clicks "Extract Concepts" (GUI) or runs concepts (CLI).
Main Success Scenario	1. The material's chunks are loaded. 2. The LLM is asked for concepts in a fixed JSON format (name, definition, source chunk). 3. The parser validates the JSON. 4. Duplicate concepts are merged. 5. Concepts are saved and displayed as concept cards.
Alternative / Exception Flows	• Malformed JSON: the request is repeated once with a format reminder, then an error is shown. • Zero concepts returned: the student is told the material may be too short.
Postconditions	Concepts are saved with the course and become the topic vocabulary for quizzes, weak-topic analysis and study plans.
Related Feature(s)	F04

UC05 — Generate Flashcards

Field	Description
-------	-------------

Use-Case ID	UC05
Name	Generate Flashcards
Actor(s)	Student, LLM Service
Goal	Create a deck of question-and-answer flashcards from a material or a chosen concept.
Preconditions	A material or concept exists.
Trigger	Student clicks "Generate Flashcards" and chooses a source and card count 5–50 (GUI), or runs flashcards generate (CLI).
Main Success Scenario	1. Relevant chunks are retrieved for the chosen source. 2. The LLM generates cards in JSON format. 3. The parser validates each card (non-empty front and back). 4. Duplicate cards are removed. 5. The student reviews, edits or deletes cards in the preview list. 6. The deck is saved to the course.
Alternative / Exception Flows	• Count outside 5–50: rejected with a message. • Fewer valid cards than requested: the student is told how many were created. • LLM failure: no partial deck is saved.
Postconditions	A flashcard deck is saved to the course.
Related Feature(s)	F05

UC06 — Review Flashcards with Spaced Repetition

Field	Description
Use-Case ID	UC06
Name	Review Flashcards with Spaced Repetition
Actor(s)	Student
Goal	Review the cards due today and reschedule each card based on how well it was remembered.
Preconditions	A deck with flashcards exists.
Trigger	Student opens "Review" mode (GUI) or runs flashcards review (CLI).
Main Success Scenario	1. The system loads the cards due today.

	- Each card is shown; the student flips it and rates recall (Again / Hard / Good / Easy). - The scheduler computes the next review interval with an SM-2 style formula. - The card's interval, ease factor and next-review date are updated and saved. - Session statistics (cards reviewed, percent correct) are recorded.
Alternative / Exception Flows	- No cards due: the date of the next due card is shown. - Session interrupted: ratings already given are kept.
Postconditions	Cards are rescheduled; the review session is recorded and feeds weak-topic analysis and the dashboard.
Related Feature(s)	F06

UC07 — Generate Quiz

Field	Description
Use-Case ID	UC07
Name	Generate Quiz
Actor(s)	Student, LLM Service
Goal	Create a practice quiz with multiple-choice and short-answer questions on chosen topics and difficulty.
Preconditions	Materials with extracted concepts exist.
Trigger	Student fills the "New Quiz" form (GUI) or runs quiz generate (CLI).
Main Success Scenario	- Chunks for the chosen topics are retrieved. - The LLM generates questions in JSON with answers and topic labels. - The parser validates each question (e.g. a multiple-choice question has exactly 4 options and one correct answer). - Invalid questions are regenerated once. - The quiz is saved and opened for taking.
Alternative / Exception Flows	- Topic not found in materials: the student is asked to choose another topic. - Not enough valid questions after regeneration: the quiz is created with fewer questions and the student is told.
Postconditions	A quiz is saved and ready to take.

Related Feature(s)	F07
--------------------	-----

UC08 — Take Quiz and Get Feedback

Field	Description
Use-Case ID	UC08
Name	Take Quiz and Get Feedback
Actor(s)	Student, LLM Service
Goal	Take a quiz and receive a score plus an explanation for every mistake.
Preconditions	A quiz exists (created via UC07).
Trigger	Student answers the questions and clicks "Submit" (GUI), or runs quiz take (CLI).
Main Success Scenario	1. Multiple-choice answers are checked deterministically against the answer key. 2. Short answers are sent to the LLM with the model answer and a grading rubric; it returns 0, 0.5 or 1 with a reason. 3. Explanations are generated for each wrong answer. 4. The score is calculated and the attempt is saved. 5. Progress statistics are updated (dashboard refreshes via the Observer).
Alternative / Exception Flows	• Blank answer: graded 0 without calling the LLM. • LLM unavailable: multiple-choice is still graded; short answers are marked 'pending' and can be graded later.
Postconditions	A QuizResult with per-question GradeResults is saved; progress trackers are notified.
Related Feature(s)	F08

UC09 — Identify Weak Topics

Field	Description
Use-Case ID	UC09
Name	Identify Weak Topics
Actor(s)	Student, LLM Service
Goal	Find the topics where the student performs worst and explain the pattern of their mistakes.
Preconditions	Quiz attempts and/or flashcard ratings exist for the

	course.
Trigger	Student opens the "Weak Topics" panel (GUI) or runs weak-topics (CLI).
Main Success Scenario	1. Accuracy is calculated per topic from attempts and ratings. 2. Topics below 70% accuracy with at least 3 attempts are selected. 3. The LLM receives the wrong answers for those topics and describes the mistake pattern. 4. The ranked list is displayed and saved for the study planner.
Alternative / Exception Flows	• Fewer than 3 attempts on every topic: the student is told to take more quizzes first. • No topic below 70%: the student is congratulated and shown their lowest topic.
Postconditions	A ranked WeakTopic list (topic, accuracy, diagnosis) is saved.
Related Feature(s)	F09

UC10 — Generate Personalized Study Plan

Field	Description
Use-Case ID	UC10
Name	Generate Personalized Study Plan
Actor(s)	Student, LLM Service
Goal	Create a day-by-day study schedule up to the exam date, giving more time to weak topics.
Preconditions	Concepts exist; weak topics are computed when attempt data is available.
Trigger	Student fills the Study Plan form: exam date, hours per day, topics (GUI), or runs plan (CLI).
Main Success Scenario	1. The agent fetches the weak topics (UC09) and the concept list. 2. The LLM drafts the plan (daily sessions: topic, activity, duration). 3. A deterministic validator checks: no day exceeds the available hours, every chosen topic appears, all dates precede the exam. 4. If validation fails, the LLM is asked to fix the listed problems (maximum 2 attempts).

	5. The plan is saved and shown in a calendar view.
Alternative / Exception Flows	• Exam date in the past: rejected immediately. • Too little time for all topics: weak topics are prioritized and the dropped topics are listed. • Validation still fails after 2 attempts: a simple deterministic plan (equal time per topic) is created instead.
Postconditions	A StudyPlan of StudyDay/StudySession entries is saved.
Related Feature(s)	F10

UC11 — View Progress Dashboard

Field	Description
Use-Case ID	UC11
Name	View Progress Dashboard
Actor(s)	Student
Goal	See per-course progress: quiz scores over time, flashcards mastered, study time, plan completion.
Preconditions	None; an empty state is handled explicitly.
Trigger	Student opens the "Dashboard" tab (GUI) or runs progress (CLI).
Main Success Scenario	1. Stored session and attempt records are loaded. 2. Statistics are computed (average score, trend, cards mastered, minutes studied, plan tasks completed). 3. The dashboard is displayed. 4. Whenever a quiz or review session ends, the dashboard receives an update notification and refreshes automatically.
Alternative / Exception Flows	• No activity yet: an empty-state message suggests a first step (upload material / generate quiz). • Corrupted record file: the bad record is skipped and the student is warned.
Postconditions	None; the dashboard is a read-only view.
Related Feature(s)	F11

UC12 — Chat with Agent (Multi-Step Task Execution and Memory)

Field	Description
-------	-------------

Use-Case ID	UC12
Name	Chat with Agent (Multi-Step Task Execution and Memory)
Actor(s)	Student, LLM Service
Goal	Let the student give requests in plain language; the agent plans the steps, calls the right tools, and remembers context.
Preconditions	Courses and materials exist (for most requests).
Trigger	Student types in the "Agent Chat" panel (GUI) or runs chat (CLI).
Main Success Scenario	1. The request, relevant memory (recent messages, remembered facts such as the exam date) and the tool list are built into a prompt. 2. The LLM returns either a tool call or a final answer. 3. The parser validates the tool call (the tool exists, arguments are valid). 4. The tool is executed and its result is added to the context. 5. Steps 2–4 repeat until a final answer is produced (maximum 10 steps). 6. The answer and the visible step list are displayed; memory is updated.
Alternative / Exception Flows	• Unknown tool or invalid arguments: the error is returned to the LLM so it can correct itself. • A tool fails: the agent reports the failure and offers an alternative instead of inventing a result. • Ambiguous request (e.g. no course given): the agent asks a clarifying question. • Step limit reached: the agent stops and summarizes what was completed.
Postconditions	Conversation messages and key facts are stored in memory for future turns.
Related Feature(s)	F12

6. Sequence Diagrams

The following sequence diagrams cover the important behaviour of the system. Each diagram references the actual classes and methods from the class diagrams in Section 3, so the dynamic view and the static view describe the same design. Closely related features that share one interaction structure (summarization, concept extraction, flashcard generation and quiz generation) are covered by a single diagram, SD-02.

SD-01 — Upload and Index Materials

Covers: UC01 / F01

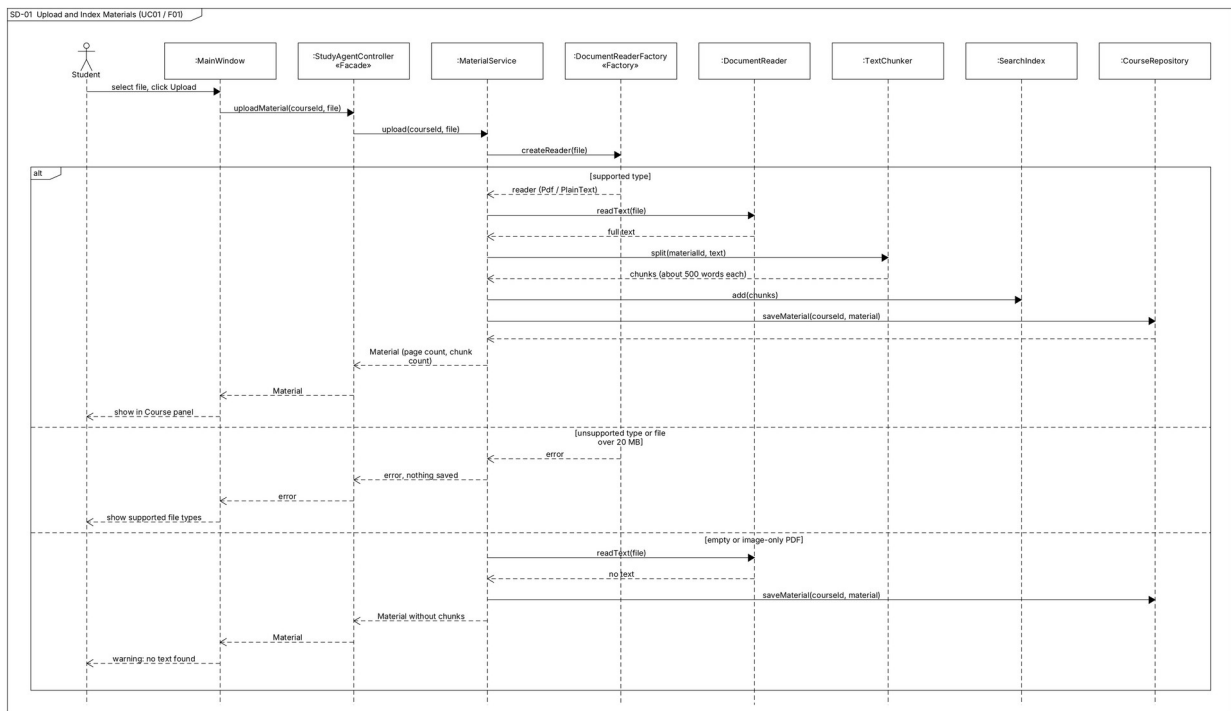
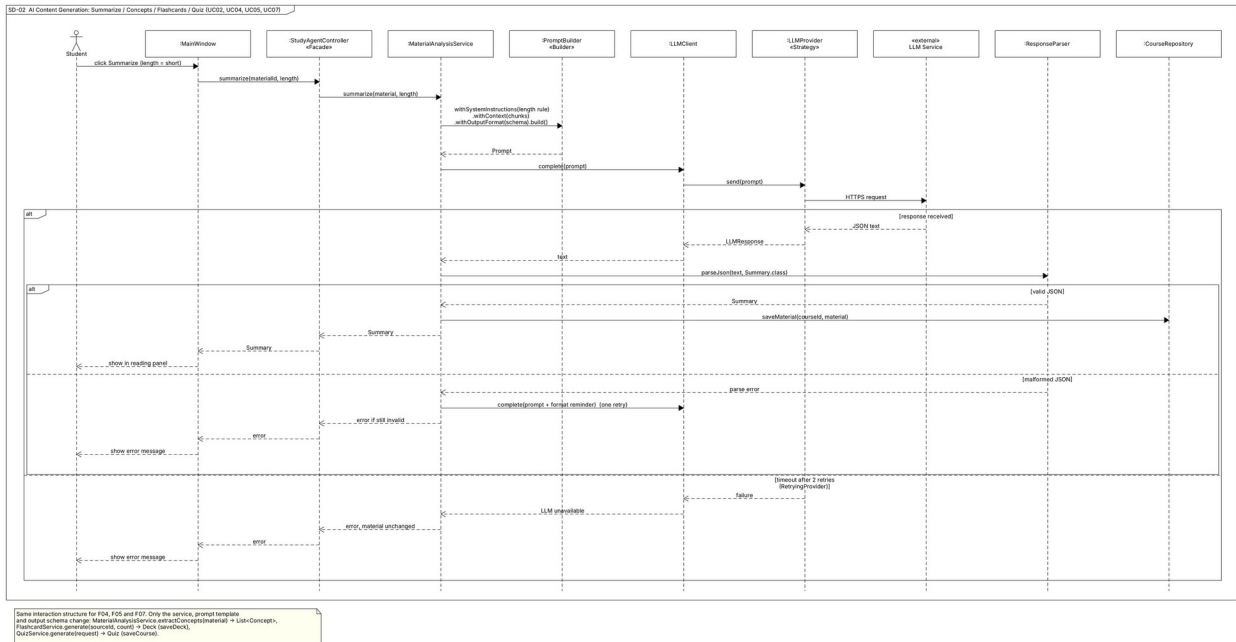


Figure SD-01 — Upload and Index Materials.

Shows the ingestion pipeline: the facade delegates to MaterialService, which uses the DocumentReaderFactory to pick a reader, extracts text, chunks it with TextChunker, indexes the chunks in SearchIndex, and persists the material through CourseRepository. Error paths are alt operands: an unsupported or oversized file returns an error and nothing is saved, while an empty or image-only PDF is saved without chunks.

SD-02 — AI Content Generation (Summarize / Concepts / Flashcards / Quiz)

Covers: UC02, UC04, UC05, UC07 / F02, F04, F05, F07



The four generation features share one interaction structure, so one diagram covers them: retrieve chunks, assemble a Prompt with PromptBuilder (system instructions, context, output schema), call the LLM through LLMClient/LLMProvider, validate the JSON with ResponseParser, then save. Only the prompt template, the output schema and the owning service (MaterialAnalysisService, QuizService, FlashcardService) differ. Retry and validation-failure paths are identical across the four.

SD-03 — Question Answering with Retrieval

Covers: UC03 / F03

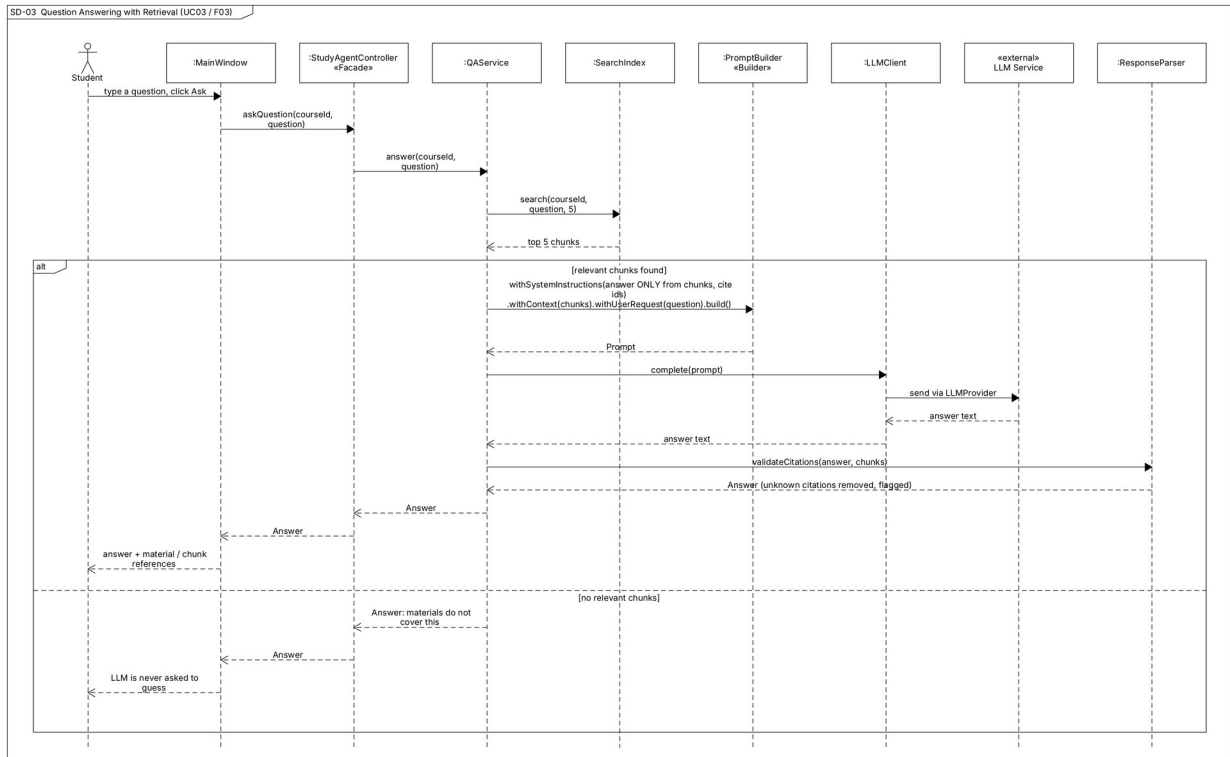


Figure SD-03 — Question Answering with Retrieval.

The hybrid retrieval pattern: deterministic keyword search (SearchIndex.search, top 5 chunks) grounds the answer, then the LLM writes it under a 'cite only these chunks' instruction, and ResponseParser.validateCitations verifies the citations. When nothing relevant is indexed, the LLM is never consulted and the system admits the gap.

SD-04 — Quiz Taking and Grading

Covers: UC08 / F08

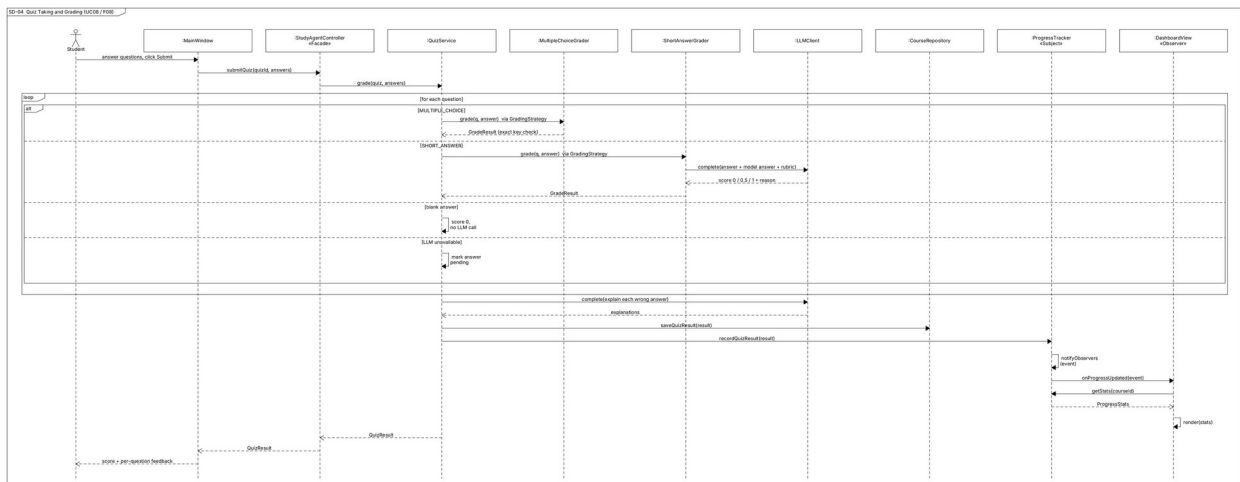


Figure SD-04 — Quiz Taking and Grading.

Demonstrates the Strategy pattern in motion: QuizService delegates each question to the GradingStrategy registered for its type — deterministic for multiple-choice, LLM-judged for short answers. The saved QuizResult flows into ProgressTracker, which notifies the DashboardView through the Observer pattern.

SD-05 — Flashcard Review with Spaced Repetition

Covers: UC06 / F06

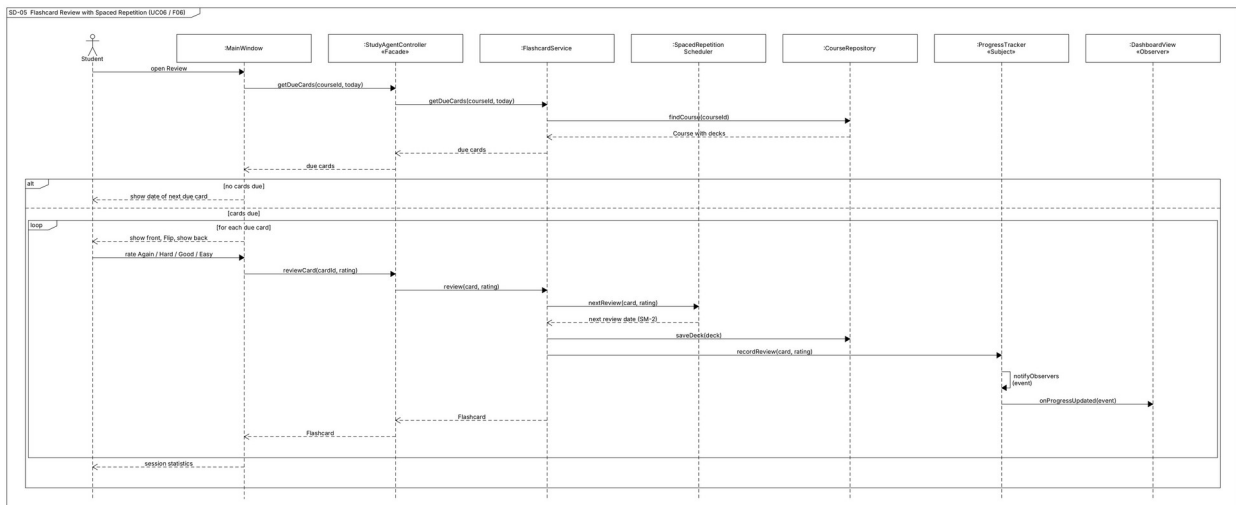


Figure SD-05 — Flashcard Review with Spaced Repetition.

A fully deterministic loop: due cards are loaded, the student's Again/Hard/Good/Easy rating drives SpacedRepetitionScheduler.nextReview (SM-2 style), and each rating is recorded with the ProgressTracker, which pushes updates to the dashboard. No LLM is involved anywhere in this flow.

SD-06 — Weak-Topic Identification

Covers: UC09 / F09

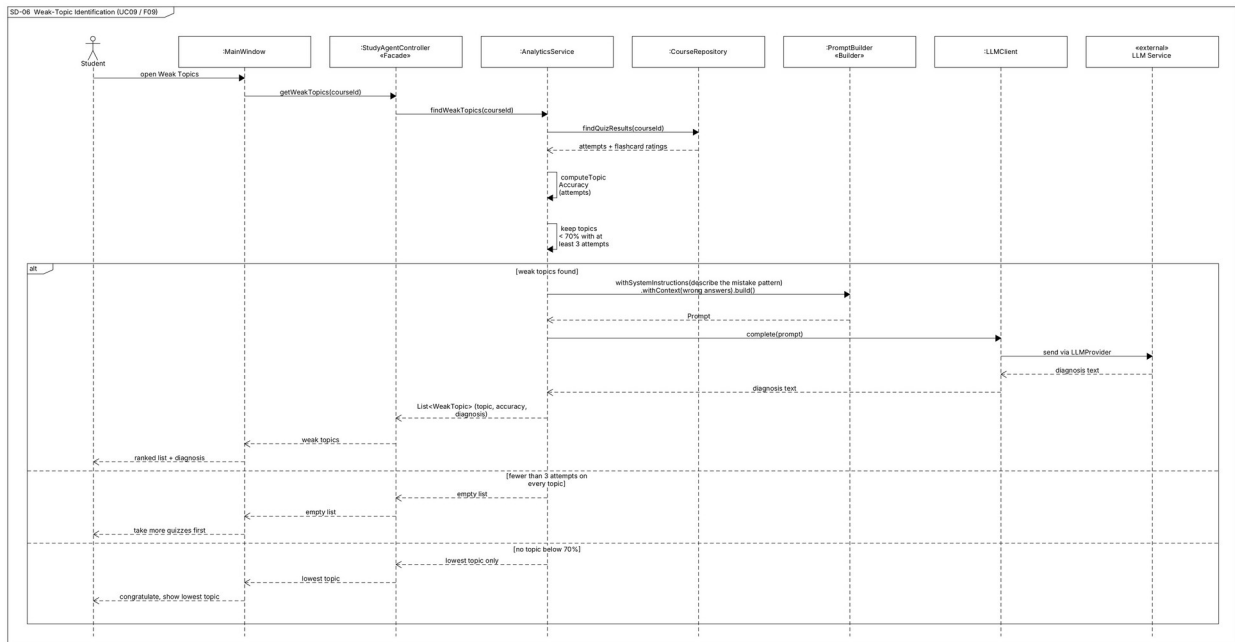


Figure SD-06 — Weak-Topic Identification.

The hybrid analysis pattern: deterministic statistics (per-topic accuracy from AnalyticsService) select candidate topics, then the LLM diagnoses the mistake pattern from the actual wrong answers. Thresholds (70% accuracy, minimum 3 attempts) guard against diagnosing noise.

SD-07 — Study Plan Generation with Validation Loop

Covers: UC10 / F10

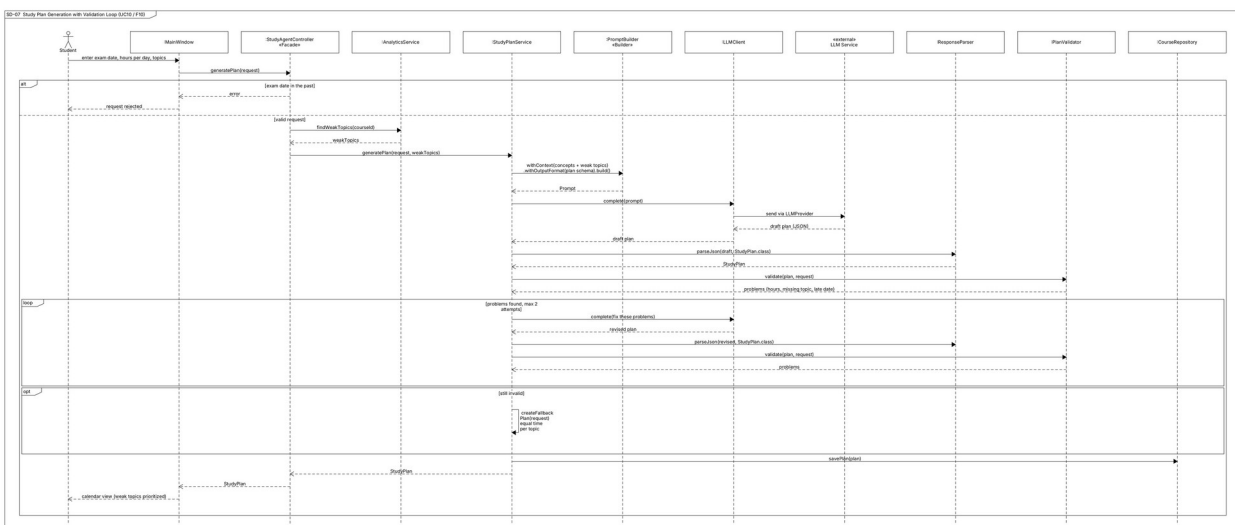


Figure SD-07 — Study Plan Generation with Validation Loop.

Shows the generate-then-validate agent collaboration: the controller first fetches weak topics from AnalyticsService, then the LLM drafts a plan from weak topics and concepts, PlanValidator checks it deterministically (hours per day, topic coverage, dates before the exam), and failures are fed back to the

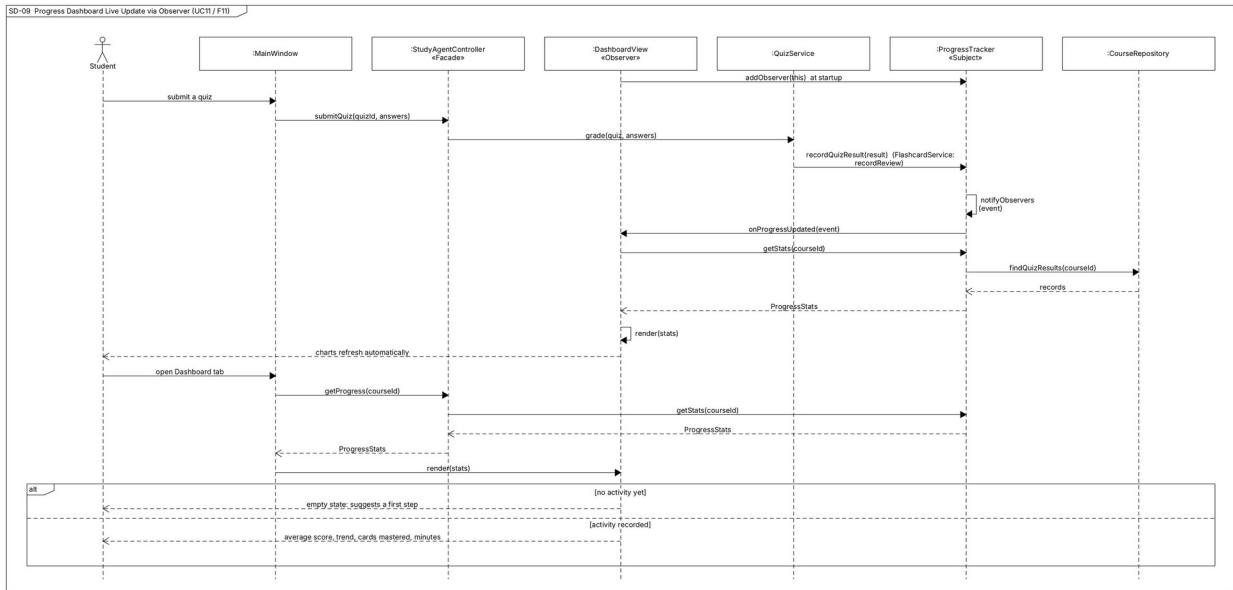


Figure SD-09 — Progress Dashboard Live Update via Observer.

The Observer pattern end to end: QuizService/FlashcardService record results with ProgressTracker (the subject) and never touch the UI; the tracker notifies registered ProgressObservers, and DashboardView re-renders. Opening the dashboard is a simple pull of the same statistics.

7. Feature-to-Design Traceability

Every feature traces to exactly one use case, the classes and key methods that implement it, the sequence diagram that shows its runtime behaviour, and the design patterns it relies on. Together with Sections 2–6, this table lets a reader start from any feature and find every design element that realizes it.

Feature	Description	Type	Use Case	Key Classes	Key Methods	Sequence Diagram	Design Pattern(s)
F01 — Course and Material Upload	Create courses; upload and index PDF/TXT/Markdown materials into searchable chunks.	Deterministic	UC01	StudyAgentController, MaterialService, DocumentReaderFactory, PdfDocumentReader, PlainTextDocumentReader, TextChunker, SearchIndex, CourseRepository	uploadMaterial, createReader, readText, split, add, saveMaterial	SD-01	Facade, Factory Method, Adapter
F02 — Material Summarization	Generate structured summaries (overview plus	AI	UC02	StudyAgentController, MaterialAnalysisService,	summarize, build, complete, send,	SD-02	Facade, Builder, Strategy, Adapter,

	key points) of a material at a chosen length.			PromptBuilder, LLMClient, LLMProvider, ResponseParser, Summary	parseJson		Decorator
F03 — Question Answering over Materials	Answer questions grounded only in uploaded materials, with chunk-level citations.	Hybrid	UC03	StudyAgentController, QAService, SearchIndex, PromptBuilder, LLMClient, ResponseParser, Answer, Chunk	askQuestion, answer, search, complete, validateCitations	SD-03	Facade, Builder, Strategy
F04 — Key Concept Extraction	Extract key concepts with one-sentence definitions as the shared topic vocabulary.	AI	UC04	StudyAgentController, MaterialAnalysisService, PromptBuilder, LLMClient, ResponseParser, Concept	extractConcepts, complete, parseJson	SD-02	Facade, Builder, Strategy
F05 — Flashcard Generation	Generate editable question/answer flashcard decks from materials or concepts.	AI	UC05	StudyAgentController, FlashcardService, PromptBuilder, LLMClient, ResponseParser, Deck, Flashcard	generateFlashcards, complete, parseJson	SD-02	Facade, Builder, Strategy
F06 — Flashcard Review (Spaced Repetition)	Spaced-repetition flashcard review scheduling by recall rating (SM-2 formula).	Deterministic	UC06	StudyAgentController, FlashcardService, SpacedRepetitionScheduler, ProgressTracker, DashboardView, Flashcard	getDueCards, reviewCard, review, nextReview, recordReview, onProgressUpdated	SD-05	Facade, Observer
F07 — Quiz Generation	Generate practice quizzes (multiple-choice plus short-answer) on chosen topics and difficulty.	AI	UC07	StudyAgentController, QuizService, PromptBuilder, LLMClient, ResponseParser, Quiz, Question	generateQuiz, generate, complete, parseJson	SD-02	Facade, Builder, Strategy
F08 — Quiz Grading and Feedback	Grade quizzes (deterministic multiple-choice, AI-	Hybrid	UC08	StudyAgentController, QuizService, GradingStrate	submitQuiz, grade, complete, recordQuizRes	SD-04	Facade, Strategy, Observer

	judged short answers) with per-mistake explanations.			gy, MultipleChoiceGrader, ShortAnswerGrader, LLMClient, QuizResult, GradeResult, ProgressTracker	ult		
F09 — Weak-Topic Identification	Rank weak topics by accuracy and diagnose mistake patterns with the LLM.	Hybrid	UC09	StudyAgentController, AnalyticsService, CourseRepository, PromptBuilder, LLMClient, WeakTopic	getWeakTopics, findWeakTopics, computeTopicAccuracy, complete	SD-06	Facade, Strategy
F10 — Personalized Study Plan	Generate validated day-by-day study plans weighted toward weak topics.	Hybrid	UC10	StudyAgentController, StudyPlanService, PlanValidator, AnalyticsService, PromptBuilder, LLMClient, ResponseParser, StudyPlan, StudyDay	findWeakTopics, generatePlan, parseJson, validate, createFallbackPlan, complete	SD-07	Facade, Builder, Strategy
F11 — Progress Dashboard	Dashboard of quiz trends, mastered cards, study time and plan completion with live updates.	Deterministic	UC11	StudyAgentController, ProgressTracker, ProgressObserver, DashboardView, ProgressStats	getProgress, recordQuizResult, recordReview, notifyObservers, onProgressUpdated, getStats, render	SD-09	Facade, Observer, Singleton*
F12 — Agent Chat (Multi-Step + Memory)	Natural-language agent chat with multi-step tool use and conversation memory.	Hybrid	UC12	StudyAgentController, AgentPlanner, MemoryManager, PromptBuilder, ResponseParser, ToolRegistry, StudyTool (+6 tools), AgentContext, AgentStep,	chat, run, decideNextAction, getRelevantContext, validateToolCall, execute	SD-08	Facade, Builder, Strategy

				AgentAction			
--	--	--	--	-------------	--	--	--

Note on F11: the Singleton (AppConfig) is listed because the dashboard's persistence path depends on the application-wide configuration (data directory) it provides.

8. Feature Implementation Explanations

This section explains, feature by feature, how the design is meant to be implemented in Stage 2: which classes collaborate, which methods carry the logic, and where AI behaviour ends and deterministic code begins. It is written as a build guide — each paragraph describes code the team will actually write.

F01 Implementation — Course and Material Upload

StudyAgentController.uploadMaterial delegates to MaterialService.upload, which asks DocumentReaderFactory for the right reader (Factory Method) and extracts text through the DocumentReader interface — PdfDocumentReader adapts Apache PDFBox internally (Adapter). TextChunker.split produces ~500-word Chunks, SearchIndex.add makes them retrievable, and CourseRepository persists the Material. The whole flow is deterministic: no LLM call exists on this path, so ingestion works offline and is trivially unit-testable with sample files.

F02 Implementation — Material Summarization

MaterialAnalysisService.summarize loads the material's chunks and builds a Prompt via PromptBuilder (withSystemInstructions for the length setting, withContext for the chunks, withOutputFormat for the summary schema). LLMClient.complete sends it through the configured LLMProvider (Strategy; wrapped in retry/logging decorators), and ResponseParser validates the returned structure before the Summary is saved. Long materials are processed chunk by chunk and the partial summaries combined, keeping every LLM call within context limits.

F03 Implementation — Question Answering over Materials

QAService.answer first calls SearchIndex.search for the top 5 chunks matching the question — a deterministic keyword retrieval step. Those chunks become the prompt context with an instruction to answer only from them. After the LLM responds, ResponseParser.validateCitations checks each citation against the retrieved chunk ids; unknown citations are stripped and the answer flagged. Grounding is therefore enforced twice: by the prompt and by the validator.

F04 Implementation — Key Concept Extraction

MaterialAnalysisService.extractConcepts reuses the SD-02 generation pipeline with a concept-extraction prompt template and a strict JSON schema (name, definition, sourceChunkId). ResponseParser.parseJson validates the structure, duplicates are merged by name, and the resulting Concepts are stored on the Course — where quizzes, weak-topic analysis and study plans later read them as the shared topic vocabulary.

F05 Implementation — Flashcard Generation

FlashcardService.generate retrieves chunks for the chosen material or concept, prompts the LLM for front/back pairs in JSON, and validates every card (non-empty sides) before de-duplicating. Crucially, cards are shown in an editable preview list first: the student confirms or fixes the AI output before the Deck is persisted, so a bad LLM run can never silently pollute the deck.

F06 Implementation — Flashcard Review with Spaced Repetition

StudyAgentController.getDueCards delegates to FlashcardService.getDueCards, which selects cards whose nextReview date is today or earlier. After each Again/Hard/Good/Easy rating, SpacedRepetitionScheduler.nextReview applies the SM-2 style formula to compute the new interval and ease factor; the card is updated and saved. Each rating is also recorded with ProgressTracker, feeding weak-topic analysis and the dashboard through the Observer — all without any AI involvement.

F07 Implementation — Quiz Generation

QuizService.generate pulls chunks for the requested topics and prompts the LLM for questions in JSON, each with four options (multiple-choice) or a model answer (short-answer) plus a topic label. ResponseParser enforces the schema — exactly 4 options and one correct answer for multiple-choice — and invalid questions get one regeneration attempt before the quiz is saved with fewer questions rather than broken ones.

F08 Implementation — Quiz Grading and Feedback

QuizService.grade iterates the questions and delegates each to the GradingStrategy registered for its type: MultipleChoiceGrader compares against the key deterministically, while ShortAnswerGrader sends the student answer, model answer and rubric to the LLM for a 0/0.5/1 score with a reason. Wrong answers get AI-generated explanations grounded in the materials. The QuizResult is saved and ProgressTracker.recordQuizResult notifies the dashboard. If the LLM is down, multiple-choice still grades and short answers wait as 'pending'.

F09 Implementation — Weak-Topic Identification

AnalyticsService.findWeakTopics aggregates QuizResults and flashcard ratings via computeTopicAccuracy into a per-topic accuracy map, then selects topics under 70% with at least 3 attempts — thresholds that prevent diagnosing noise. The wrong answers for those topics are sent to the LLM with a 'describe the mistake pattern' instruction, producing the human-readable diagnosis stored on each WeakTopic for the dashboard and study planner.

F10 Implementation — Personalized Study Plan Generation

StudyAgentController first calls AnalyticsService.findWeakTopics (F09) and passes the result to StudyPlanService.generatePlan, which adds the course concepts and has the LLM draft a schedule of StudyDay/StudySession entries. PlanValidator.validate then checks the draft deterministically: daily minutes within the student's availability, every requested topic covered, all dates before the exam.

Failures are returned to the LLM as concrete repair instructions (max 2 rounds); if validation still fails, `createFallbackPlan` builds a simple equal-time schedule so the student always receives a valid plan.

F11 Implementation — Progress Tracking Dashboard

`ProgressTracker` is the single funnel for progress events: `QuizService` and `FlashcardService` call `recordQuizResult` and `recordReview`, which persist the records and notify observers. `DashboardView`, registered as a `ProgressObserver`, re-renders via `getStats/render` on every event — no polling, no service-to-UI calls. `AppConfig` (Singleton) supplies the data directory used by the JSON repository underneath. The CLI progress command reads the same statistics, so both front-ends agree.

F12 Implementation — Agent Chat with Multi-Step Task Execution and Memory

`StudyAgentController.chat` hands the request to `AgentPlanner.run`, which loops: `MemoryManager.getRelevantContext` injects recent messages and remembered facts, `PromptBuilder` assembles the prompt with tool descriptions, the LLM returns an `AgentAction` (`TOOL_CALL`, `FINAL_ANSWER`, `ASK_CLARIFICATION`), `ResponseParser.validateToolCall` checks it against the `ToolRegistry`, and the chosen `StudyTool` executes against the real services. Each step is recorded in the `AgentContext` (max 10 steps), invalid calls are fed back to the LLM for self-correction, and `MemoryManager` stores the outcome — giving the agent planning, tool use, validation, and short- and long-term memory.